

Dokumentacja projektu Symulacja Miasta

Projekt: SymulacjaMiasto | C++17 | CMake | Google Test

Apolonia Sowa, Justyna Patan, Jakub Mularczyk

1. Opis działania

Konsolowa symulacja średniowiecznego miasta. Gracz obserwuje rozwój osady turami i steruje tempem gry (pojedyncza tura lub fast-forward). Celem jest utrzymanie miasta przez gromadzenie zasobów, rozwój infrastruktury i przetrwanie losowych zdarzeń.

- Zasoby: FOOD, GOLD, WOOD, STONE oraz SAFETY (0–100).
- Wzorce: Factory, Manager, polimorfizm (Citizen, Building, RandomEvent).
- Przegrana: 3 kolejne tury z $\text{FOOD} \leq 0$, $\text{GOLD} \leq 0$ lub $\text{SAFETY} \leq 0$.

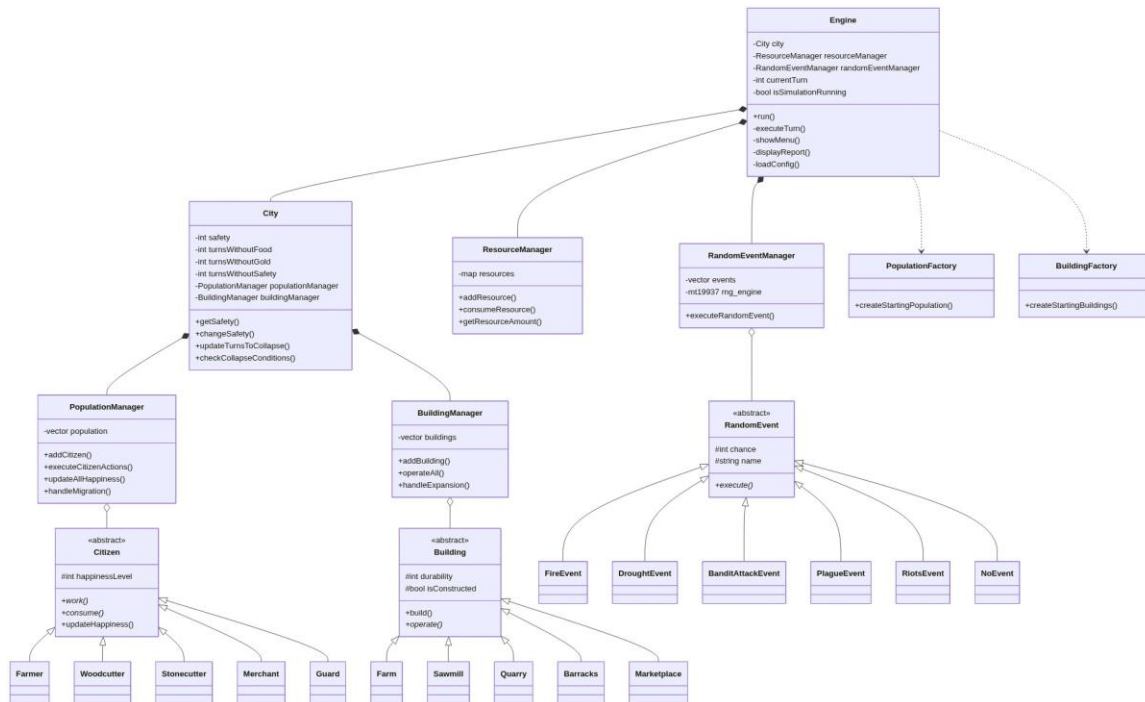
2. Przebieg tury

- Krok 1: `PopulationManager::executeCitizenActions()` — `work()` i `consume()` każdego obywatela
- Krok 2: `BuildingManager::operateAll()` — produkcja budynków
- Krok 3: `PopulationManager::updateAllHappiness()` — aktualizacja szczęścia
- Krok 4: `PopulationManager::handleMigration()` — co 5. turę ($\text{FOOD}/\text{czł.} \geq 5$)
- Krok 5: `BuildingManager::handleExpansion()` — co 10. turę, losowy budynek
- Krok 6: Konsumpcja globalna: -25 FOOD , -18 GOLD
- Krok 7: `RandomEventManager::executeRandomEvent()` — od tury 6
- Krok 8: `City::updateTurnsToCollapse()` i `checkCollapseConditions()`

3. Inicjalizacja

- `loadConfig('config.txt')` lub wartości domyślne
- `PopulationFactory` — 10 obywateli startowych
- `BuildingFactory` — 5 budynków startowych

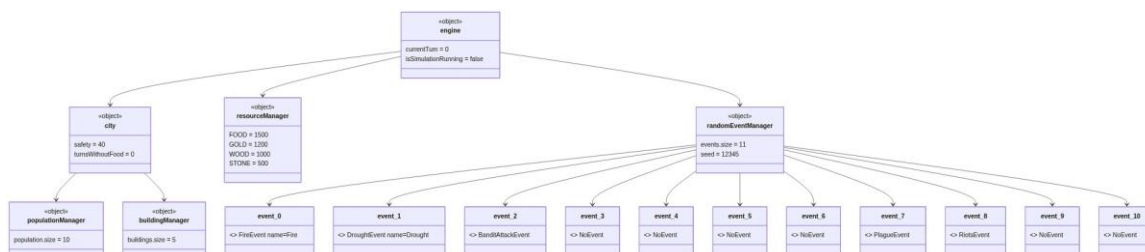
4. Diagram klas



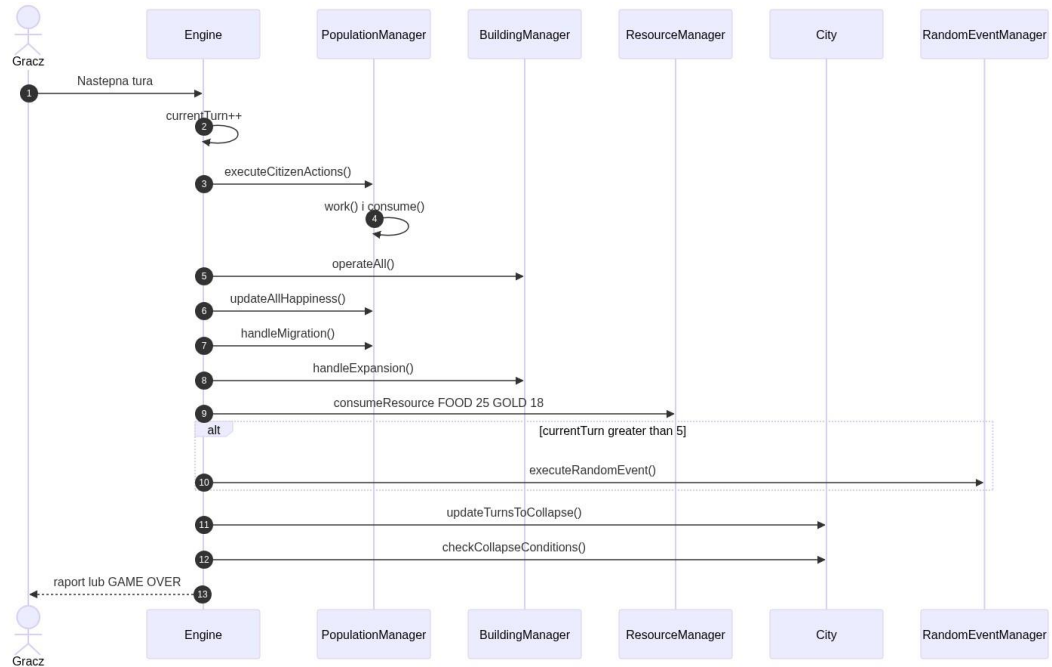
5. Diagram obiektów

Migawka po inicjalizacji. RandomEventManager zawiera 11 instancji:

Indeks	Typ	name
0	FireEvent	Fire
1	DroughtEvent	Drought
2	BanditAttackEvent	Bandits Attack
3	NoEvent	nothig :)
4	NoEvent	nothig :)
5	NoEvent	nothig :)
6	NoEvent	nothig :)
7	PlagueEvent	Plague
8	RiotsEvent	Riots
9	NoEvent	nothig :)
10	NoEvent	nothig :)



6. Diagram sekwencji (jedna tura)



7. Opis klas — pola i metody

Engine

Pola: city, resourceManager, randomEventManager, currentTurn, isSimulationRunning

Metody: public: Engine(), run() | private: executeTurn(), showMenu(), displayReport(), loadConfig(), trim()

Silnik gry — menu, pętla tur, raporty, ładowanie config.txt.

City

Pola: safety, turnsWithoutFood/Gold/Safety, populationManager, buildingManager

Metody: getSafety(), changeSafety(), updateTurnsToCollapse(), checkCollapseConditions(), getPopulationManager(), getBuildingManager()

Stan miasta i warunki upadku.

ResourceManager

Pola: resources (mapa ResourceType → int)

Metody: addResource(), consumeResource(), getResourceAmount()

PopulationManager

Pola: population (vector<unique_ptr<Citizen>>)

Metody: addCitizen(), executeCitizenActions(), updateAllHappiness(),
getAverageHappiness(), getPopulation(), handleMigration()

BuildingManager

Pola: buildings (vector<unique_ptr<Building>>)

Metody: addBuilding(), operateAll(), handleExpansion()

RandomEventManager

Pola: events, rng_engine (seed 12345)

Metody: addEvent(), executeRandomEvent(), executeAllEvents()

PopulationFactory

Pola: —

Metody: createStartingPopulation(city, rm) — 5 Farmer, 2 Merchant, 1 Woodcutter, 1
Stonecutter, 2 Guard

BuildingFactory

Pola: —

Metody: createStartingBuildings(bm, city, rm) — Farm, Sawmill, Quarry, Barracks,
Marketplace

Citizen (abstrakcyjna)

Pola: happinessLevel | Metody: work()*, consume()*, updateHappiness(), setHappiness(),
getHappiness(), getProductionMultiplier(), scaledProduction()

Farmer: pola resourceManager*, foodProduction, foodConsumption | +FOOD / -FOOD

Woodcutter: pola resourceManager*, woodProduction, foodConsumption | +WOOD /
-FOOD

Stonecutter: pola resourceManager*, stoneProduction, foodConsumption | +STONE /
-FOOD

Merchant: pola resourceManager*, foodConsumption | +GOLD (gdy FOOD>50) / -FOOD

Guard: pola city*, resourceManager*, safetyProduction, foodConsumption | +SAFETY /
-FOOD

Building (abstrakcyjna)

Pola: durability, maxDurability, isConstructed, constructionCost | Metody: build(),
takeDamage(), operate()*, getName()*, getDurability(), getIsConstructed()

Farm: operate() +15 FOOD | koszt: 20 WOOD, 10 STONE

Sawmill: operate() +10 WOOD | koszt: 5 WOOD, 10 STONE

Quarry: operate() +10 STONE | koszt: 15 WOOD, 5 STONE

Barracks: operate() +10 SAFETY | koszt: 20 WOOD, 20 STONE

Marketplace: operate() +8 GOLD | koszt: 15 WOOD, 15 STONE

RandomEvent (abstrakcyjna)

Pola: chance, description, name | Metody: execute(), getName(), getDescription(), getChance()

FireEvent: -30% WOOD, -25 SAFETY

DroughtEvent: -40% FOOD

BanditAttackEvent: -GOLD (skalowane SAFETY), -40 SAFETY

PlagueEvent: -15 SAFETY, -20% FOOD

RiotsEvent: -GOLD (skalowane SAFETY), -25 SAFETY

NoEvent: brak efektu

ResourceType (enum)

FOOD, GOLD, WOOD, STONE

Logger

logMessage(msg) — zapis do simulation_logs.txt | clearLogFile() — czyszczenie przy starcie

8. Warunki końca gry

- $\text{FOOD} \leq 0$ przez 3 tury $\rightarrow$ NOT ENOUGH FOOD
- $\text{GOLD} \leq 0$ przez 3 tury $\rightarrow$ NOT ENOUGH GOLD
- $\text{SAFETY} \leq 0$ przez 3 tury $\rightarrow$ IT AINT SAFE